# [Khanh Pham] C programming review

# C Programming Review

## Function

### Passing by Value

In C, function arguments are **always passed by value**.

This means that when we pass a variable to a function, the function receives a **copy of its value**.

For example, after all, a still have the value 5 because the variable which pass to the function just a copy.

```
void change(int x)
{
    x = 10;
}

int main()
{
    int a = 5;
    change(a);
}
```

If we want a function to modify the original variable in memory, we can pass the variable's address to the function.

We use the & operator to get the address and the * operator to dereference the pointer. For example:

```
int swap(int *a, int *b)
{
    *a = *a ^ *b;
    *b = *a ^ *b;
    *a = *a ^ *b;
}

int main()
{
    int a = 5;
    int b = 6;
    swap(&a, &b);
    printf("%d",a);
}
```

# Function prototype

In C, the compiler needs to know a function's name, return type, and parameter types before the function is called.

If the function definition appears after the function call, we can declare a function prototype before the call.

For example:

```
#include<stdio.h>

int swap(int *a, int *b);

int main()
{
    int a = 5;
    int b = 6;
    swap(&a, &b);
    printf("%d",a);
}
```

```
int swap(int *a, int *b)
{
    *a = *a ^ *b;
    *b = *a ^ *b;
    *a = *a ^ *b;
}
```

Without a declaration visible before the call, the compiler may not know the function's correct signature. In larger programs, function prototypes are commonly placed in header files `.h`, while the actual function definitions are placed in source files `.c`.

# Pointer and Array

## Pointer Arithmetic

When we use `+` or `-` with a pointer, the pointer moves by the size of the data type it points to.

For example:

```
int arr[5];
int *p = arr;
```

If an `int` occupies 4 bytes:

```
p       → address of arr[0]
p + 1   → address of arr[1]
p + 2   → address of arr[2]
```

The address does not increase by only 1 byte.

If we want to increse each byte, we need to convert to (char*) type.

---

## Pointer to Pointer

A **pointer to pointer** is a pointer that stores the address of another pointer.

For example:

```
int x = 10;
int *p = &x;
int **pp = &p;
```

We can access the values at different levels:

```
x    → 10
*p   → 10
*pp  → value of p (address of x)
**pp → 10
```

A pointer to pointer is useful when a function needs to **modify a pointer itself**, rather than only modify the value that the pointer points to.

For example:

```
void insert(link_list **head, int data)
{
    if ((*head) == NULL)
    {
        link_list *temp = malloc(sizeof(link_list));
        temp->next = NULL;
        temp->data = data;
        (*head) = temp;
        return;
    }
    else
    {
        link_list *cur = *head;
        while (cur->next != NULL)
        {
            cur = cur->next;
        }

        link_list *temp = malloc(sizeof(link_list));
        temp->next = NULL;
        temp->data = data;
        cur->next = temp;
    }
}
```

Here, `int **head` allows the function to modify the original pointer of singly linklist.

# Const of pointer

When we use keyword **`const`** for pointer, there are 2 case:

- `const` for value: `const int *a`

```
int main()
{
    int b = 5;
    const int *a;
    a = &b;
}
```

We cannot use pointer `a` to change the value of `b`.

- `const` for address: `int * const a`

```
int main()
{
    int b = 5;
    int c = 8;
    int * const a = &b;
}
```

We cannot point `a` to another variable like `c` beacuse a just only point to `b`.

# Pointer casting and Endianess

Endianess is the way how to memory store. There are two case:

- Big endian: The most significant byte goes to the lowest memory address
- Litte edian: The least significant byte goes to the lowest memory address.

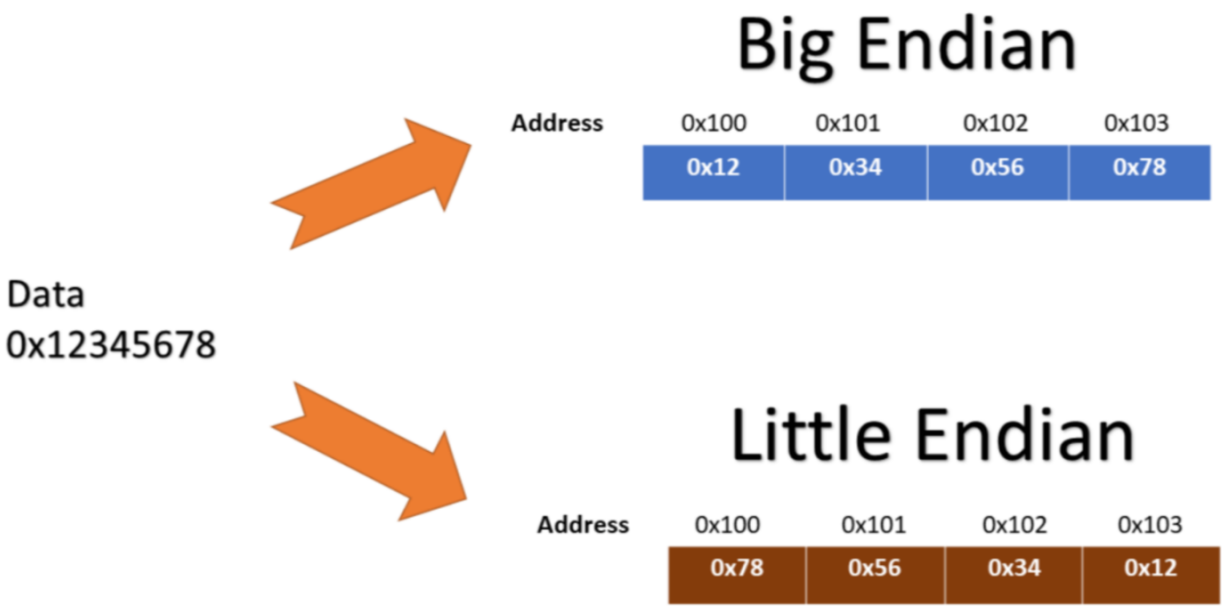
If we want to check any system is big or litte endian, we can casting the pointer which point to any datatype that is higher than 1 byte to `char*` and print the value to check: For example:

```
#include <stdio.h>
```

```
int main()
{
    int num = 0x12345678;
    char *a = (char *)&num;
    printf("%x", *a);
}
```

If the result is `78`, the system is litte endian

If the result is `12`, the system is big endian




# Wild pointer

Wild pointer is the situation that when we create a pointer without pointing to any address. Therefore, this pointer will point to NULL pointer or trash address. So we can't interact with the memory that pointer is pointing to, if we interact, the program will crash.

```
#include <stdio.h>

int main()
{
    int *a;
    printf("%d",*a);
}
```

if we run the program, the result is: `Segmentation fault (core dumped)`

# Buffer overflow

This case occur when a program writes more data into an array or memory block than it can hold. For example:

```
int main()
{
    int a[5];

    for (int i = 0; i <= 5; i++)
    {
        a[i] = 0;
    }
}
```

**The program will not crash, the compiler will not alert error too**, however, this action will overwrite other memory, so it can make the program wrong

# Struct, Union, Typedef

## Struct

A structure (`struct`) is a collection of one or more variables, possibly of different types, grouped together under a single name for convenient handling.

For example:

```
struct Student
{
    int id;
    char grade;
    double score;
};
```

A structure allows us to group related data of different types into a single object.

---

# Size of struct

The memory layout of a structure contains:

1. The memory required by its members.
2. **Padding bytes** added by the compiler to satisfy alignment requirements.

Therefore:

```
sizeof(struct) = size of all members + padding
```

However, the compiler does **not** simply add padding randomly. Each member must be placed at an address that satisfies its **alignment requirement**.

# Alignment

Each data type has an **alignment requirement**.

For example, on a typical system:

```
char   → alignment = 1
short  → alignment = 2
int    → alignment = 4
double → alignment = 8
```

If a member requires 4-byte alignment, its address must be a multiple of 4.

The compiler inserts padding when necessary to ensure that each member is correctly aligned.

---

# Tail Padding

The compiler may also add padding **at the end of a structure**.

For example:

```
struct S2
{
    int i;
    int j;
    char c;
};
```

The members require:

```
4 + 4 + 1 = 9 bytes
```

However, the compiler may make the structure size 12 bytes:

Therefore:

```
sizeof(struct S2) = 12 bytes
```

The final 3 bytes are called **tail padding**.

# Why is tail padding necessary?

Consider an array:

```
struct S2 arr[4];
```

If `sizeof(struct S2)` were only 9 bytes, the elements would start at:

```
arr[0] → x
arr[1] → x + 9
arr[2] → x + 18
arr[3] → x + 27
```

If the structure requires 4-byte alignment, only the first element may be correctly aligned.

By making the structure size 12 bytes:

```
arr[0] → x
arr[1] → x + 12
arr[2] → x + 24
arr[3] → x + 36
```

and:

```
12 % 4 = 0
```

Therefore, every element of the array can satisfy the structure's alignment requirement.

---

# Rule for calculating the size of struct

## Rule 1: The adress index must is a multiple of element alignment

**Purpose**: improving memory system performent

For example, an int element must stay in adress 0,4,8,...

## Rule 2: The total size of struct must be multiple of the highest alignment in all elements. (In other word, struct need tail padding)

```
struct test
{
□char a;
□int b;
□char c;
};
```

In the example, follow the first rule, each element stay in suitable position of them like this:

| element | address index | size + padding |
|---|---|---|
| a | 0 | 1 + 3 |
| b | 4 | 4 |
| c | 8 | 1 |

Afer that, the size of struct is 9 bytes

Follow the second rule, the total of struct is 9, the highest alignment is 4, so we need add 3 bytes of padding to the size of struct. After all, size of struct is 9 + 3 = 12 bytes

# Rule when creating a struct

Because padding in struct, we must follow a rule that keep the order of all elements in struct from the smallest to the biggest( or from biggest to smallest is same).

```
struct test
{
    char a;
    int b;
    char c;
};
```

Result:

```
Size of struct is 12 bytes
```

When we follow the rule:

```
struct test {
    char a;
    char c;
    int b;
};
```

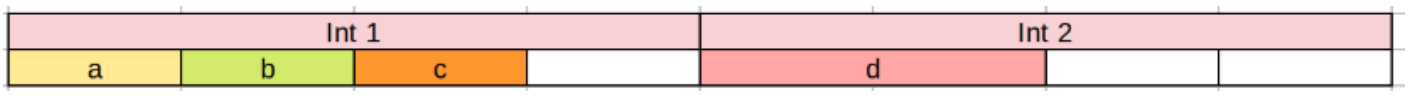
Result

```
Size of struct is 8 bytes
```

# Bit field in struct

In struct, Bitfield allows structure members to use a specific number of bits instead of the full size of their data type.

```
struct test
{
    int a : 8;
    int b : 8;
    int c : 8;
    int d : 16;
}
```

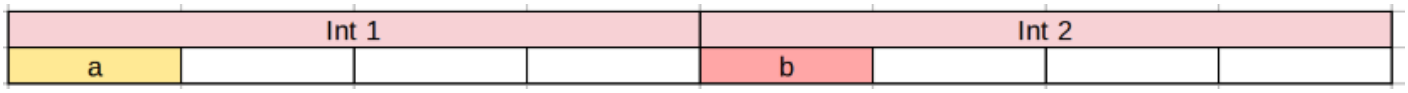



The size of above struct is `8 bytes`. When we create the first `int` element, in memory, 4 bytes will be allocated. Howerver, because of using bitfield, 3 first `int` elements just only use 8 bits each. Therefore, the total of all 3 element is 24 bits, that mean less than 32 bit of 4 bytes that was allocated. With the last `int` element, because it requires 16 bits, more than 8 bits that is empty in last memory, so this element will be stayed in the next int memory, that mean 4 bytes next.

When we want to make **the next element do not use the memory of the last element**, we can use: `int : 0;` in middle of two elements:

```
struct test
{
  int a : 8;
  int : 0;
  int b : 8;
};
```

With this example, although there are 2 elements with total bits is 16 bits. However, because we use `int : 0`, so the second element cannot use the memory of the first element.




# Union

A **union** is a user-defined data type in C that allows multiple members of **different data types** to share the **same memory location**.

For example:

```
union Number
{
    int i;
    double d;
};
```

A variable of type `union Number` can store either an `int` or a `double`:

```
union Number p;


p.i = 10;
```

or:

```
p.d = 3.14;
```

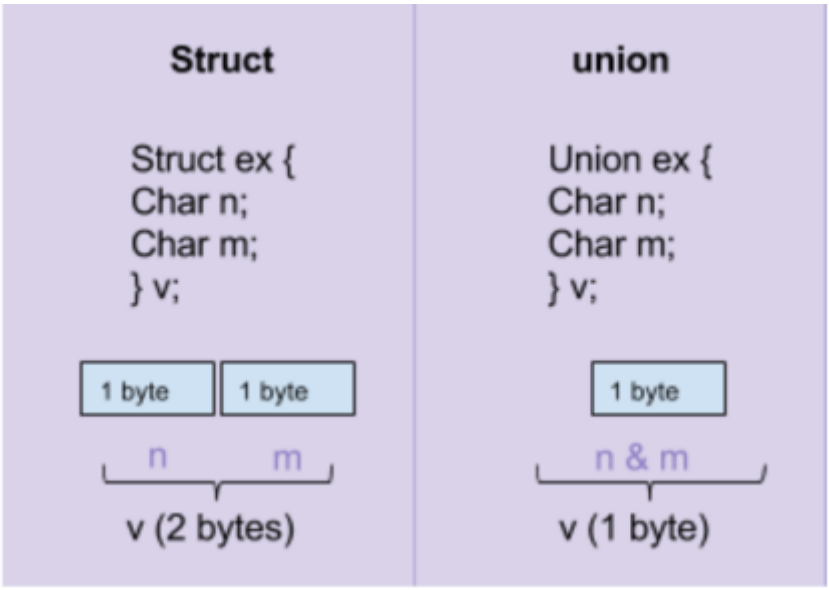
However, the members **share the same memory**, so a union can hold the value of **only one member at a time** in the usual use case.

---

## Size of a Union

The size of a union is large enough to hold its **largest member**, including any alignment requirements.

## Struct vs Union




# Typedef

Typedef` is a C keyword used to create an **alias** for an existing data type.

It does **not create a new data type**. Instead, it gives an existing type another name that can be used when declaring variables, functions, structures...

```
typedef struct student
{
    int height;
    int age;
} learner;



int main()
{
    learner a;
    a.height = 167;
    a.age = 15;
}
```

# Dynamic allocation

## Why Do We Need Dynamic Memory?

With a normal fixed-size array, its size cannot be changed after it has been created.

For example:

```
int arr[10];
```

The array can store exactly 10 `int` values. We cannot directly resize it to 20 elements.

This can be inefficient:

- If we allocate too little memory, the array may not be large enough.
- If we allocate too much memory, some of the allocated memory may never be used.

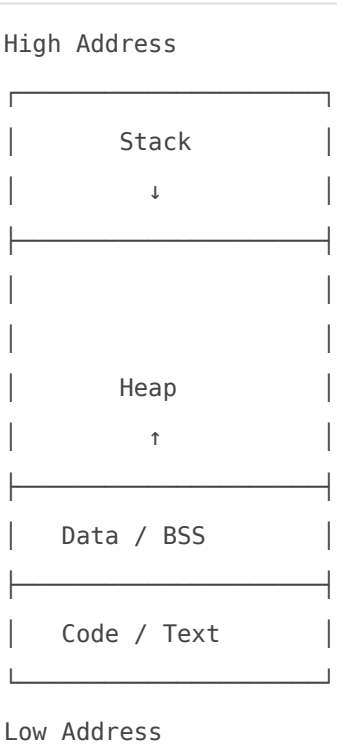
**Dynamic memory allows us to allocate memory based on the actual requirements at runtime.**

---

# Heap Memory

In a typical process memory layout, dynamically allocated memory comes from an area called the **heap**.

A simplified process memory layout is:




The heap is used for memory that is allocated dynamically during program execution.

---

# `malloc()`

`malloc()` allocates a block of memory from the heap. this function returns void* pointer if allocation success, NULL if failed

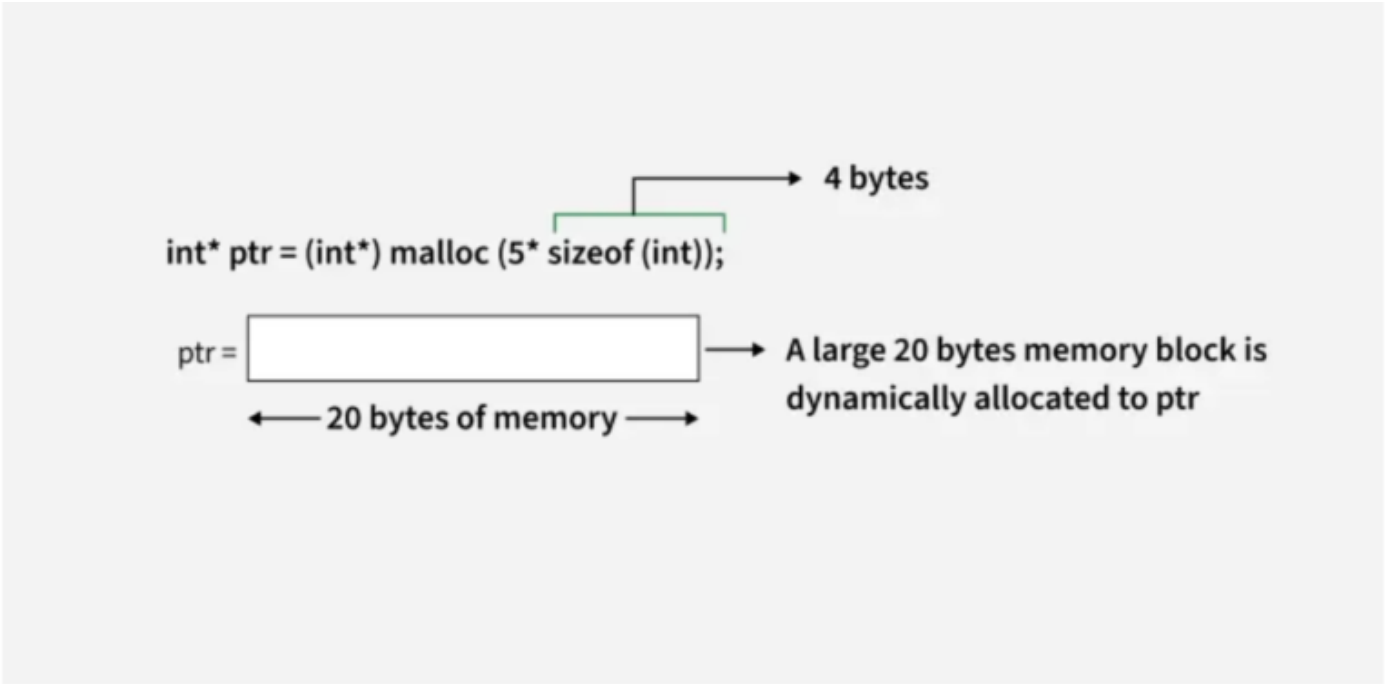
Its prototype is:

```
void *malloc(size_t size);
```

It takes the number of **bytes** to allocate and returns a pointer to the allocated memory.

For example:

```
int *p = malloc(5 * sizeof(int));
```




---

# `calloc()`

`calloc()` allocates memory for multiple elements **and initializes all allocated bytes to zero.**

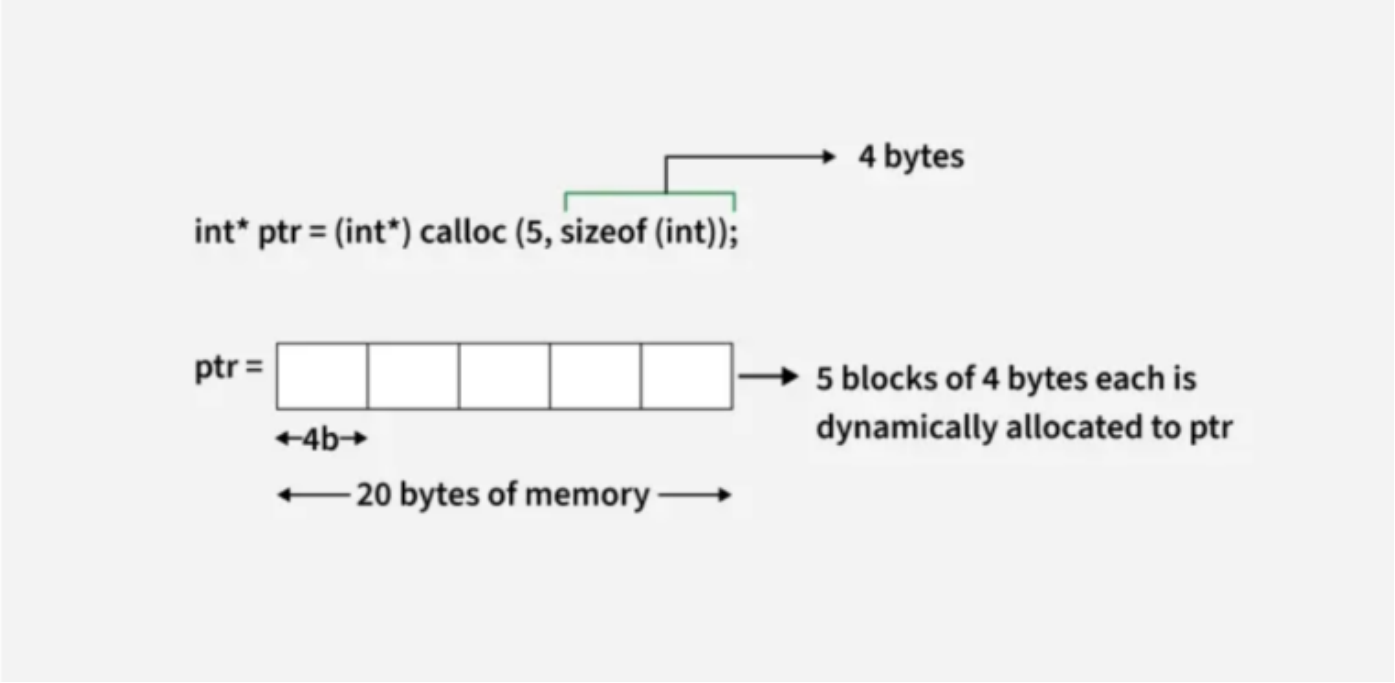
Its prototype is:

```
void *calloc(size_t n, size_t size);
```

For example:

```
int *p = calloc(5, sizeof(int));
```

This allocates enough memory for 5 `int` values.




---

# `realloc()`

`realloc()` changes the size of a previously allocated memory block. it return NULL if fail

Its prototype is:

```
void *realloc(void *ptr, size_t size);
```

For example:

```
int *p = malloc(5 * sizeof(int));


p = realloc(p, 10 * sizeof(int));
```

This requests that the allocation be resized to hold 10 `int` values.

Conceptually:

```
Before:


┌────┬────┬────┬────┬────┐
|  1 |  2 |  3 |  4 |  5 |
```

```
└────┴────┴────┴────┴────┘


After:

┌────┬────┬────┬────┬────┬────┬────┬────┬────┬────┐
│  1 │  2 │  3 │  4 │  5 │ ?  │ ?  │ ?  │ ?  │ ?  │
└────┴────┴────┴────┴────┴────┴────┴────┴────┴────┘
```

The existing contents are preserved up to the smaller of the old and new sizes.

> **Important:** `realloc()` may move the memory block to a different address. Therefore, the pointer returned by `realloc()` should be used. Older block memory was released

A safer pattern is:

```
int *tmp = realloc(p, 10 * sizeof(int));

if (tmp != NULL) {
    p = tmp;
}
```

This prevents losing the original pointer if `realloc()` fails.

---

# `free()`

`free()` releases dynamically allocated memory.

Its prototype is:

```
void free(void *ptr);
```

For example:

```
int *p = malloc(5 * sizeof(int));

/* use p */
```

```
free(p);
```

After `free(p)`, the allocated memory is no longer available for the program to use.

A useful practice is:

```
free(p);
p = NULL;
```

Setting the pointer to `NULL` helps prevent accidentally using the old pointer value.

---

# Memory Leak

A **memory leak** occurs when dynamically allocated memory is no longer reachable by the program but has not been released with `free()`.

For example:

```
void function()
{
    int *p = malloc(100 * sizeof(int));

    /* Do something without using `free` to release memory which p is managing */
}
```

When the function ends, the local pointer `p` disappears, but the allocated heap memory may still be allocated.

The program can no longer access that memory through `p`.

If this happens repeatedly:

```
allocate → lose pointer → allocate → lose pointer → ...
```

the process may consume an increasing amount of memory.

---

# Dangling Pointer

A **dangling pointer** is a pointer that refers to memory that is no longer valid or has already been released.

For example:

```
int *p = malloc(sizeof(int));
*p = 10;
free(p);
```

After `free(p)`, `p` still contains the old address, but that memory is no longer valid for access. If we access to this memory, this is the Undefined Behavior, however, **the program will not crash**, so it will make the mistake of the program's logic. Therefore, we should point to NULL after releasing.

```
free(p);
p = NULL;
```

Now `p` no longer contains the old address.

---

# Double Free

A **double free** occurs when the same dynamically allocated memory is released more than once.

For example:

```
int *p = malloc(sizeof(int));

free(p);
free(p);
```

Calling `free()` on the same allocation twice results in undefined behavior. **The programm will be crashed and returned the error double free in runtime**. Therefore, we also should point to NULL after free.

```
free(p);
p = NULL;
```

After that, if we forget and free p again, it's ok because free NULL is safe

```
free(p);
```

# Compilation process

Compilation is the process that convert from source code to the execute file that can be run in the machine.

| Step | Purpose | Command to see target file |
| --- | --- | --- |
| Preprocessor | Delete all user comments, Replace #include keyword by their source, Replace all macros by their definition | gcc -E `.c file` -o `.i file` |
| Compiler | Transfer from higher language to assembly code. | gcc -S `.i file` -o `.s file` |
| Assembler | Transfer from assembly code to machine code | gcc -c `.s file` -o `.o file` |
| Linker | Transfer from higher language to assembly code. | gcc -S `.o file` -o `.s file` |

# Parameters in main function

In main function, we can pass parameters from shell when we run the program by:

```
int main(int argc, char* argv[])
```

`argc` is the number of parameter that we pass to the function

`argv` is an array of pointers to the arguments

Example:

```
#include <stdio.h>

int main(int argc, char* argv[])
{
	for (int i = 0; i < argc; i++)
        {
              printf("the %d parameter is: %s\n", i + 1, argv[i]);
	}
}
```

The result when we run by this command `./test khanhne 123` is:

```
the 1 parameter is: ./test
the 2 parameter is: khanhne
```

```
the 3 parameter is: 123
```

# lab practice

```
https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/C_programming_review
```

---

Revision #19
Created 2026-08-19 03:25:25 UTC by Khanh Pham
Updated 2026-08-20 10:30:57 UTC by Khanh Pham